

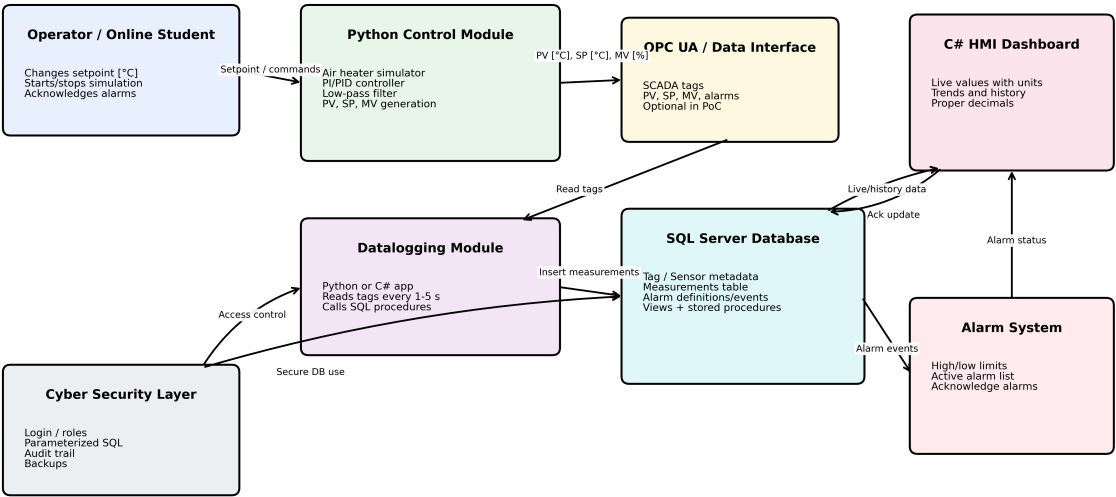
Design and Implementation of a Modular SCADA Prototype for Simulated Air-Heater Control

Python OPC UA Control, SQL Server Datalogging, C# HMI and Alarm Management

Author: Rahaf Hashem

Date: May 7, 2026

System Sketch: Simulated Modular SCADA System for Air Heater Control



The paper presents a modular SCADA prototype with OPC UA communication, SQL Server datalogging, alarm handling, HMI visualization and cybersecurity analysis.

Contents

1	Introduction	2
1.1	Use of Artificial Intelligence	2
2	System Overview and Architecture	2
3	Methods and Implementation	3
3.1	Database Design	3
3.2	Control System	3
3.3	OPC UA Communication and Datalogging	3
3.4	Alarm System	4
3.5	Human–Machine Interface	4
4	Results and Evaluation	5
4.1	Control Response	5
4.2	Functional Verification	5
5	Cybersecurity Considerations	5
5.1	Attack Surface	5
5.2	Zones, Conduits and Security Levels	6
5.3	Implemented and Recommended Controls	6
6	Discussion	6
7	Conclusion	6
A	Implementation Files and Key Interfaces	8
A.1	OPC UA Tag Interface	8
A.2	Database Interface	8
A.3	OPC UA Datalogging Interface	8
A.4	HMI Alarm Acknowledgement Interface	8

Abstract

This paper presents the design and implementation of a modular Supervisory Control and Data Acquisition (SCADA) prototype for simulated air-heater temperature control. The final architecture consists of a Python-based air-heater simulation and PI controller exposed as an OPC UA server, a Python OPC UA client used as a datalogging module, a SQL Server database, an alarm management subsystem, and a C# WinForms Human–Machine Interface (HMI). A first-order process model is used to represent the air-heater dynamics, while a discrete PI controller and a low-pass filter are implemented from scratch. Process variables such as temperature, filtered temperature, setpoint, control signal and control error are exposed as OPC UA tags and stored in SQL Server with timestamps, engineering units and quality information. The alarm subsystem detects abnormal conditions, stores alarm events and supports operator acknowledgement. The results demonstrate a complete end-to-end SCADA workflow from OPC UA process communication to database storage, HMI visualization, alarm monitoring and historical analysis. Cybersecurity considerations are discussed using IEC 62443, NIST CSF 2.0 and NIS2.

Keywords— SCADA, OPC UA, Python, SQL Server, C#, PI control, datalogging, alarm system, HMI, IEC 62443, cybersecurity.

1 Introduction

Supervisory Control and Data Acquisition (SCADA) systems are used to monitor, control and acquire data from industrial processes. Typical SCADA systems contain process control, data acquisition, database storage, alarm handling and operator interfaces. The project specification requires a modular SCADA prototype including a database, control system, datalogging system, alarm system and cybersecurity discussion [1].

A central requirement in a SCADA-oriented architecture is support for standardized communication between modules. OPC is commonly used for this purpose, and the final prototype therefore includes OPC UA between the control module and the datalogging module. SQL Server is used for historical storage, alarm history and HMI queries.

This work presents a software-based SCADA prototype for an air-heater process. The objective is to demonstrate a complete software architecture where independent modules exchange process data, store historical information, generate alarms and support operator monitoring. The prototype is implemented using Python, OPC UA, SQL Server and C# WinForms.

The main contribution is a compact SCADA prototype that combines process simulation, OPC UA communication, SQL Server datalogging, alarm handling and HMI visualization in one integrated workflow. The focus is on software modularity, traceability and practical validation of the main SCADA functions.

1.1 Use of Artificial Intelligence

AI-based tools were used to support requirement interpretation, architecture planning, language refinement,

code organization and LaTeX structuring. AI assistance was also used to review possible design alternatives, such as direct database communication versus OPC UA-based communication. All generated suggestions were reviewed, modified and validated during implementation. The final technical decisions, screenshots, tests and system integration were performed by the author

2 System Overview and Architecture

Figure 1 shows the complete system architecture. The prototype follows a modular software design where each module has a defined responsibility and communicates through clear interfaces.

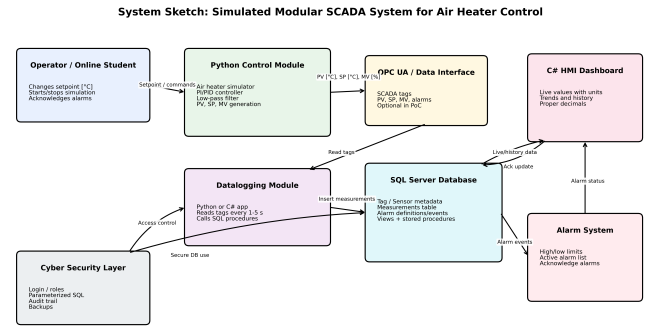


Figure 1: System architecture of the modular SCADA prototype.

The Python control module simulates the air-heater process, calculates the PI control signal and exposes process variables as OPC UA tags. A separate Python datalogging module acts as an OPC UA client, reads the tags and writes the values to SQL Server. SQL Server stores tag metadata, time-series measurements, alarm definitions and alarm events. The C# HMI reads the latest values and active alarms from SQL Server and allows alarm acknowledgement.

This architecture separates live process communication from historical storage. OPC UA is used for process data exchange, while SQL Server is used for persistence, alarm history and HMI queries. This follows a more industrial SCADA-style structure than direct database communication from the control module.

Table 1: Software modules and responsibilities.

Module	Responsibility
Python OPC UA Server	Simulates the air-heater process, computes PI control output and exposes process tags.
Python OPC UA Client / Logger SQL Server	Reads OPC UA tags and inserts measurements into SQL Server. Stores tags, measurements, alarm definitions and alarm events.
C# HMI	Displays live values and alarms and supports acknowledgement.
Alarm Logic	Detects abnormal states and stores alarm events.

3 Methods and Implementation

3.1 Database Design

The database was implemented in SQL Server using a tag-based SCADA structure. The purpose of the database is to store process metadata, time-series values, alarm definitions, alarm events and OPC UA tag information. The design separates metadata from measurement data, which makes the system flexible and extensible.

Figure 2 shows the entity-relationship diagram. The **Tag** table stores metadata for process variables. The **Measurement** table stores timestamped values. The **AlarmDefinition** table stores alarm limits and priorities. The **AlarmEvent** table stores generated alarms and acknowledgement information.

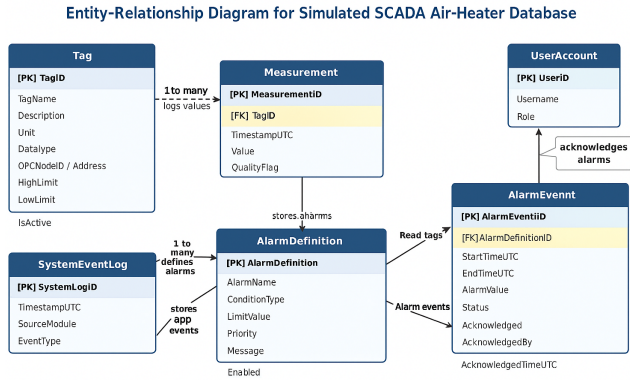


Figure 2: Entity-relationship diagram for the SCADA database.

The main process tags are:

- **AirHeater.Temperature** [°C]
- **AirHeater.FilteredTemperature** [°C]
- **AirHeater.Setpoint** [°C]
- **AirHeater.ControlSignal** [%]
- **AirHeater.Error** [°C]

OPC UA metadata was also added to the tag structure. This includes the OPC UA node name, namespace and server endpoint. Figure 3 shows the OPC UA tag metadata stored in SQL Server.

	TagName	Unit	OPCNodeName	OPCNamespace	OPCServerEndpoint
1	AirHeater.ControlSignal	%	ControlSignal	http://rahaf-scada-airheater	opc.tcp://localhost:4840/scada/airheater/
2	AirHeater.Error	°C	Error	http://rahaf-scada-airheater	opc.tcp://localhost:4840/scada/airheater/
3	AirHeater.FilteredTemperature	°C	FilteredTemperature	http://rahaf-scada-airheater	opc.tcp://localhost:4840/scada/airheater/
4	AirHeater.Setpoint	°C	Setpoint	http://rahaf-scada-airheater	opc.tcp://localhost:4840/scada/airheater/
5	AirHeater.Temperature	°C	Temperature	http://rahaf-scada-airheater	opc.tcp://localhost:4840/scada/airheater/

Figure 3: OPC UA metadata stored in the SQL Server tag table.

Stored procedures are used as stable interfaces between applications and the database. This reduces duplicated SQL in the application layer and improves maintainability. The implementation files and selected interfaces are summarized in Appendix A.

3.2 Control System

The process is represented as a simplified first-order air-heater model. The simulated temperature depends on ambient temperature, process gain, time constant and

control signal. The discrete process model is expressed as:

$$T_{k+1} = T_k + \frac{\Delta t}{\tau} (-(T_k - T_{amb}) + K u_k) \quad (1)$$

where T_k is the process temperature, T_{amb} is ambient temperature, K is process gain, τ is the time constant and u_k is the control signal.

A low-pass filter is applied to reduce measurement noise:

$$PV_f(k) = \alpha PV(k) + (1 - \alpha) PV_f(k - 1) \quad (2)$$

where PV_f is the filtered process value, PV is the measured temperature and α is the filter coefficient.

The PI controller is implemented as:

$$e(k) = SP(k) - PV_f(k) \quad (3)$$

$$MV(k) = K_p e(k) + K_i \sum e(k) T_s \quad (4)$$

where SP is the setpoint, MV is the manipulated variable, K_p is the proportional gain, K_i is the integral gain and T_s is the sample time. The controller output is limited to 0–100%.

A PI controller was selected instead of a full PID controller because the air-heater represents a slow thermal process with dominant first-order dynamics. Proportional action gives immediate correction, while integral action removes steady-state error. Derivative action was not included because it is sensitive to measurement noise and could introduce unnecessary control signal variation.

Table 2: Simulation and PI controller parameters.

Parameter	Value	Unit
Sampling time, T_s	1.0	s
Temperature setpoint, SP	50.0	°C
Ambient temperature, T_{amb}	22.0	°C
Proportional gain, K_p	3.0	–
Integral gain, K_i	0.08	s ⁻¹
Control signal limits	0–100	%

The selected parameters in Table 2 were sufficient to maintain stable temperature control during the evaluated test.

3.3 OPC UA Communication and Data-logging

The final implementation uses OPC UA between the control module and the datalogging module. The Python control module runs as an OPC UA server and exposes air-heater variables as OPC UA tags. A separate Python datalogging module acts as an OPC UA client and reads the tags at a fixed interval before inserting the values into SQL Server.

Figure 4 shows the OPC UA server and OPC UA datalogger running in parallel. The server produces the simulated process values, and the datalogger confirms that values are logged from OPC UA.

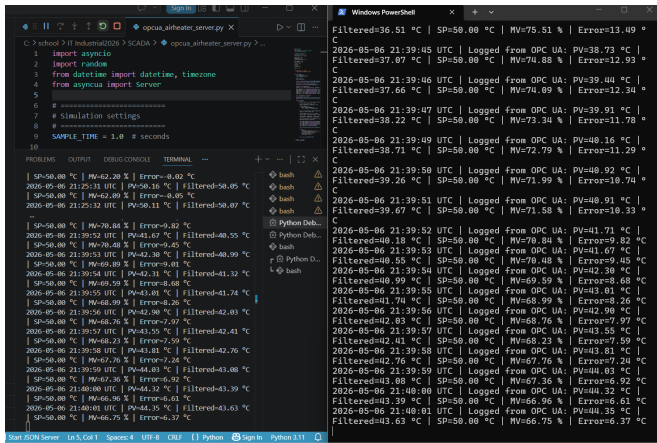


Figure 4: OPC UA server and OPC UA datalogging module running in parallel. The Python control module exposes process tags, while the datalogging client reads the tags and logs them to SQL Server.

The logger uses one-second sampling during testing. Each record is stored with tag name, timestamp, value, unit and quality flag. Figure 5 shows OPC UA-logged values retrieved from SQL Server.

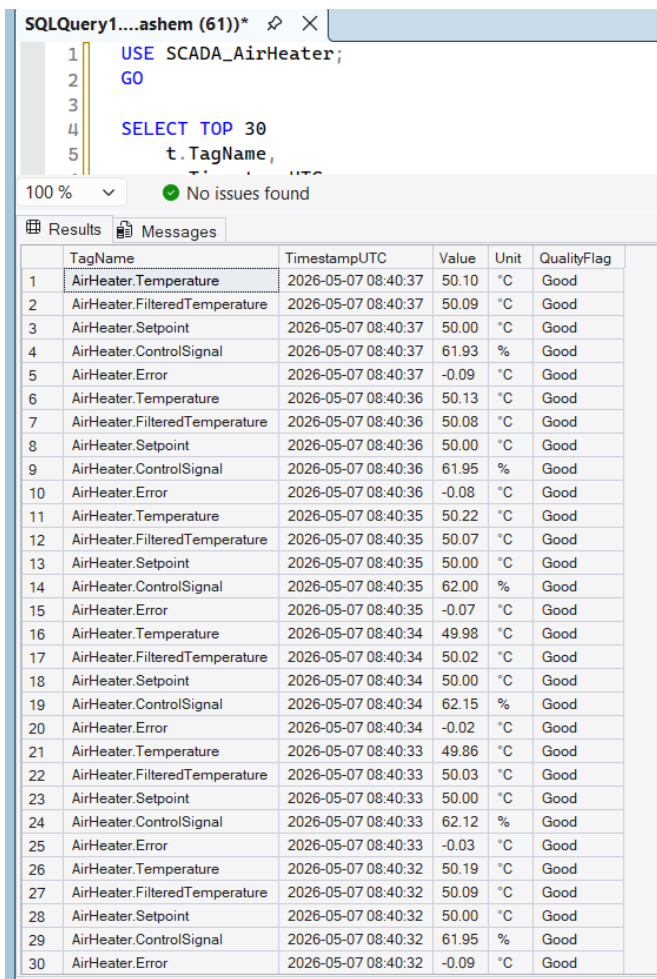


Figure 5: SCADA measurements logged from OPC UA into SQL Server with units and quality flags.

3.4 Alarm System

The alarm subsystem detects abnormal conditions and stores alarm events in SQL Server. Three alarm cases

are included in the prototype:

- High temperature alarm.
- Low temperature alarm.
- Controller saturation alarm.

The alarm philosophy follows common SCADA practice: an alarm can be active, acknowledged and cleared. The database stores the alarm value, alarm status, acknowledgement state, acknowledgement user and acknowledgement timestamp. During validation, the high-temperature alarm limit was temporarily lowered to force an alarm condition and verify the complete alarm workflow.

Figure 6 shows both the active high-temperature alarm and acknowledgement evidence. The alarm is visible in the HMI, and the acknowledgement status, operator name and timestamp are stored in SQL Server.

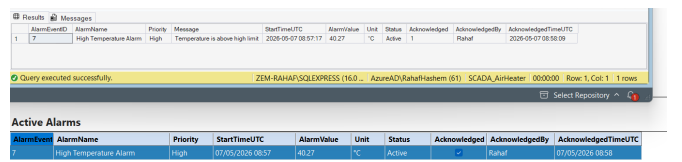


Figure 6: Active high-temperature alarm and acknowledgement evidence. The alarm is displayed in the HMI, and acknowledgement information is stored in SQL Server.

This validation confirms that the alarm system supports both alarm visibility and operator traceability. The alarm condition was generated from process data, displayed in the HMI and acknowledged through the operator interface. The database record then preserved the acknowledgement status, user name and timestamp, which is important for later audit and troubleshooting.

3.5 Human–Machine Interface

The HMI was implemented using C# WinForms. Figure 7 shows the final HMI with OPC UA-logged data. The HMI displays live process values, active alarms, acknowledgement status and control buttons. Values are shown with engineering units and suitable decimal precision.

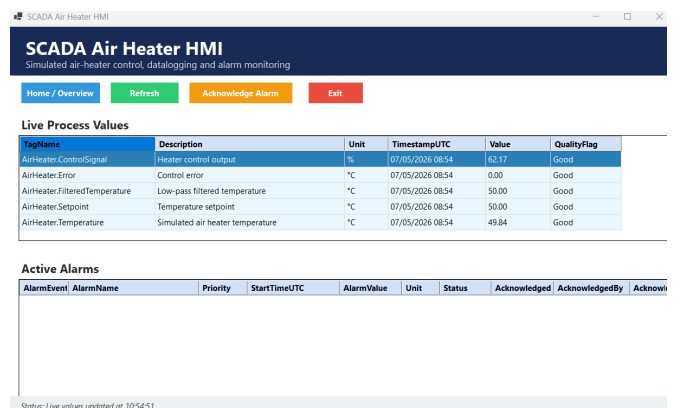


Figure 7: C# SCADA HMI showing process values and active alarms based on OPC UA-logged data.

The HMI reads latest values using a stored procedure and reads active alarms from a database view. When an

alarm is acknowledged, the HMI calls the acknowledgement stored procedure. This creates traceability between operator action and database state.

4 Results and Evaluation

The complete system was evaluated by running the Python OPC UA server, starting the OPC UA datalogger, verifying SQL Server inserts, opening the C# HMI and triggering alarm acknowledgement. The validated data flow is:

Python OPC UA server → OPC UA client/logger → SQL Server → C# HMI → alarm acknowledgement.

4.1 Control Response

Figure 8 shows the startup and steady-state response of the simulated air-heater process using OPC UA datalogging. At the beginning of the test, the measured temperature is below the 50 °C setpoint. The PI controller responds by applying a high control signal to increase the simulated heating power. As the measured and filtered temperatures approach the setpoint, the controller output decreases and stabilizes.

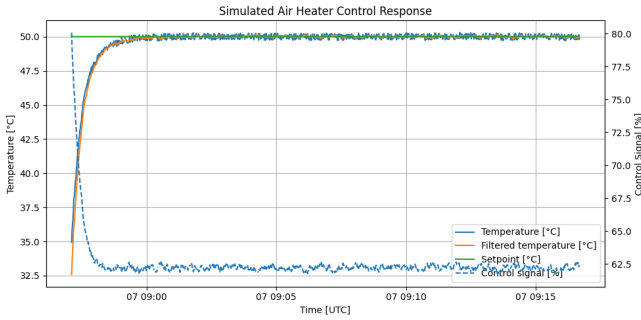


Figure 8: Startup and steady-state response of the simulated air-heater process using OPC UA datalogging. The measured temperature rises toward the 50 °C setpoint, while the filtered temperature provides a smoother signal. The control signal starts high during warm-up and settles around 62%.

The result shows stable setpoint tracking after the startup transient. The small fluctuations in the measured temperature are expected because simulated sensor noise was added to the process value. The filtered temperature is smoother and therefore more suitable for control and monitoring. Since the control signal stabilizes instead of oscillating, the selected PI parameters are considered suitable for the simulated process.

The result also supports the choice of PI control instead of PID control. The integral action removes steady-state error, while the absence of derivative action avoids amplification of measurement noise in the temperature signal. For this slow thermal process, the PI controller provides sufficient performance without the additional complexity of derivative control.

4.2 Functional Verification

Table 3 maps the implemented prototype to the main SCADA requirements.

Table 3: Coverage of main SCADA requirements.

Requirement	Implemented solution
Database	SQL Server database with tags, measurements, OPC UA metadata, alarm definitions and alarm events.
Control system	Python air-heater simulation with PI controller and low-pass filter.
OPC UA communication	Python OPC UA server exposes air-heater tags, and a Python OPC UA client reads the tags.
Datalogging	OPC UA client inserts timestamped values into SQL Server using stored procedures.
Alarm system	Active alarms are generated, stored, displayed and acknowledged.
HMI	C# WinForms HMI displays live values and alarm states from SQL Server.
Cybersecurity	Attack surface, IEC 62443 zones, security levels and improvements are discussed.

The results show that the prototype satisfies the required SCADA workflow. The control module generates process values, OPC UA exposes the values as tags, the datalogging module stores values in SQL Server, the database preserves history, the alarm module records abnormal states, and the HMI provides operator interaction.

5 Cybersecurity Considerations

Cybersecurity is important in SCADA systems because manipulation of process values, control signals, alarms or operator actions may affect availability, integrity and safety. The prototype contains typical SCADA assets: an OPC UA control interface, database, datalogger, HMI and alarm subsystem.

5.1 Attack Surface

The main attack surfaces are:

- OPC UA communication between the control module and datalogger.
- SQL Server access and permissions.
- HMI interaction and alarm acknowledgement.
- Application code and dependency vulnerabilities.
- Loss of logger availability or database availability.

The IEC 62443 secure-by-design framework identifies communication attacks, local control/gateway attacks and central platform attacks as important attack paths for energy-management control systems [2]. These paths map directly to this prototype: OPC UA communication compromise can affect data exchange, control-module compromise can affect generated values, and database/HMI compromise can affect displayed values and alarms.

5.2 Zones, Conduits and Security Levels

IEC 62443-3-2 recommends defining the system under consideration, partitioning it into zones and conduits, assessing risk and assigning security level targets. IEC 62443-3-3 defines technical requirements linked to the foundational requirements of the standard [2, 3].

For this prototype, the logical zones are:

- Zone 1: Python OPC UA server and control module.
- Zone 2: OPC UA datalogger and SQL Server database.
- Zone 3: C# HMI and operator interface.

The OPC UA connection and database connection are the main conduits between the zones. The implemented prototype corresponds approximately to Security Level 1–2 because it includes basic authentication through Windows Authentication, stored procedures and alarm traceability. However, the OPC UA implementation does not yet use certificate-based trust, encryption or user authentication. In the implemented prototype, the OPC UA conduit is functional but not yet hardened. Therefore, the cybersecurity evaluation distinguishes between implemented controls, such as SQL Server authentication and alarm traceability, and recommended controls, such as OPC UA certificates and network segmentation.

5.3 Implemented and Recommended Controls

Implemented controls include Windows Authentication for SQL Server, stored procedures, parameterized database commands, quality flags and alarm acknowledgement traceability.

Recommended improvements include:

- Role-based access control for operator, engineer and administrator roles.
- Least-privilege SQL accounts.
- OPC UA encryption and certificate-based authentication.
- Network segmentation between control, database and HMI zones.
- Centralized logging and monitoring.
- Backup and recovery procedures.
- Patch and vulnerability management.

These recommendations align with IEC 62443 secure-by-design principles, where segmentation, communication security, host security and cybersecurity monitoring are treated as design concerns rather than afterthoughts [2]. NIST CSF 2.0 also supports a structured view of cybersecurity through Govern, Identify, Protect, Detect, Respond and Recover [4]. NIS2 is relevant as a European cybersecurity reference because it emphasizes risk management, incident handling, supply-chain security and management accountability [5].

6 Discussion

The prototype demonstrates that the main SCADA functions can be implemented as separate software modules while still forming one coherent system. The most important design decision was to separate live process communication from historical storage. OPC UA was used between the simulated control module and the datalog-

ging module, while SQL Server was used for persistence, alarm history and HMI queries. This separation makes the architecture more realistic than a solution where the control module writes directly to the database.

During development, direct database communication was useful for early validation because it made the database, alarm tables and HMI easier to test. However, the final OPC UA-based approach is more appropriate for a SCADA prototype because process values are exposed as standardized tags and consumed by a separate client. This improves modularity: the control module does not need to know how measurements are stored, and the datalogging module can be modified without changing the process simulation.

The control response shows that the simulated air-heater reaches the 50 °C setpoint and remains close to this operating point after the startup transient. The filtered temperature follows the measured temperature with less noise, which confirms the purpose of the low-pass filter. The control signal decreases from a high initial value and settles around 62%, indicating that the PI controller reaches a stable operating point. For this slow thermal process, PI control was sufficient because the integral action removed steady-state error while derivative action would increase sensitivity to measurement noise.

The alarm test verifies another important SCADA function: operator traceability. The high-temperature alarm was generated, displayed in the HMI and acknowledged by the operator. The acknowledgement status, operator name and timestamp were stored in SQL Server. This is important because alarm acknowledgement is not only a user-interface action; it is also an audit event that should be stored for later review.

From a software engineering perspective, the solution is maintainable because each module has a clear responsibility. The OPC UA server handles process values, the datalogger handles acquisition and storage, SQL Server handles persistence, and the HMI handles operator interaction. This reduces coupling between modules and makes the prototype easier to extend with real hardware or additional tags.

The main limitation is that the process model is simplified. The air-heater is represented as a first-order thermal model, which is sufficient for validating the SCADA workflow but not for detailed physical analysis. A second limitation is that OPC UA security is not fully configured. The current implementation demonstrates functional OPC UA communication, but a production-oriented system should use certificates, encrypted communication, user authentication and network segmentation. Finally, the HMI provides live monitoring and alarm acknowledgement, but future work should add integrated trend plotting and role-based access control.

7 Conclusion

A modular SCADA prototype for simulated air-heater control was designed, implemented and evaluated. The final system includes OPC UA communication between the Python control module and the Python datalogger, SQL Server storage for measurements and alarms, and a

C# HMI for monitoring and alarm acknowledgement.

The results show that the PI controller maintains the simulated process close to the 50 °C setpoint, while the low-pass filter reduces measurement noise. The OPC UA datalogging test verifies that process tags can be acquired through a standard SCADA communication protocol and stored in SQL Server. The alarm test verifies that alarm events can be generated, displayed, acknowledged and stored with operator traceability.

The prototype satisfies the main SCADA requirements and provides a practical foundation for further development. The most important future improvements are OPC UA security configuration, role-based access control, integrated HMI trending, more detailed process modelling and testing with physical hardware.

A Implementation Files and Key Interfaces

This appendix summarizes the implementation files used to reproduce the prototype. The complete source files are submitted together with the PDF. Only the most relevant interfaces are listed here to keep the appendix compact and readable.

Table 4: Implementation files.

File	Language	Purpose
scada_database.sql	SQL	Creates SCADA tables, tags, OPC UA metadata and alarm definitions.
scada_procedures.sql	SQL	Defines procedures for logging, history retrieval and acknowledgement.
scada_views.sql	SQL	Defines views for live values and active alarms.
opcua_airheater_server.py	Python	Implements simulation, PI control, filtering and OPC UA server tags.
opcua_sql_datalogger.py	Python	Reads OPC UA tags and logs values to SQL Server.
plot_scada_results.py	Python	Generates the control-response plot from SQL Server data.
Program.cs	C#	Implements the WinForms HMI.

A.1 OPC UA Tag Interface

```
1 airheater = await objects.add_object(idx, "AirHeater")
2
3 node_temperature = await airheater.add_variable(idx, "Temperature", temperature)
4 node_filtered = await airheater.add_variable(idx, "FilteredTemperature", filtered_temperature)
5 node_setpoint = await airheater.add_variable(idx, "Setpoint", setpoint)
6 node_control = await airheater.add_variable(idx, "ControlSignal", 0.0)
7 node_error = await airheater.add_variable(idx, "Error", 0.0)
8
9 await node_setpoint.set_writable()
```

Listing 1: Core OPC UA tag interface in the Python server.

A.2 Database Interface

```
1 CREATE PROCEDURE dbo.InsertMeasurement
2     @TagName NVARCHAR(100),
3     @Value DECIMAL(18,4),
4     @QualityFlag NVARCHAR(20) = 'Good'
5 AS
6 BEGIN
7     DECLARE @TagID INT;
8
9     SELECT @TagID = TagID
10    FROM dbo.Tag
11   WHERE TagName = @TagName AND IsActive = 1;
12
13    INSERT INTO dbo.Measurement (TagID, Value, QualityFlag)
14    VALUES (@TagID, @Value, @QualityFlag);
15 END;
```

Listing 2: Core database interface for inserting measurements.

A.3 OPC UA Datalogging Interface

```
1 for tag_name, node in nodes.items():
2     value = await node.read_value()
3     values[tag_name] = value
4     insert_measurement(cursor, tag_name, value)
5
6 temperature = values["AirHeater.Temperature"]
7 control_signal = values["AirHeater.ControlSignal"]
8
9 check_alarms(cursor, temperature, control_signal)
```

Listing 3: Core OPC UA datalogging logic.

A.4 HMI Alarm Acknowledgement Interface

```
1 using SqlConnection conn = new SqlConnection(connStr);
2 using SqlCommand cmd = new SqlCommand("dbo.AcknowledgeAlarm", conn);
3
4 cmd.CommandType = CommandType.StoredProcedure;
5 cmd.Parameters.AddWithValue("@AlarmEventID", alarmEventId);
6 cmd.Parameters.AddWithValue("@AcknowledgedBy", "Rahaf");
7
```

```
8 | conn.Open();  
9 | cmd.ExecuteNonQuery();  
10  
11 | RefreshAlarms();
```

Listing 4: C# HMI call for alarm acknowledgement.

References

- [1] H.-P. Halvorsen, “Scada systems assignment,” 2026, course assignment handout, University of South-Eastern Norway.
- [2] G. B. Gaggero, A. Armellin, P. Girdinio, and M. Marchese, “An iec 62443-based framework for secure-by-design energy communities,” *IEEE Access*, 2024, author accepted version provided as course/resource material.
- [3] International Society of Automation, “ISA/IEC 62443 Series of Standards,” <https://www.isa.org/standards-and-publications/isa-standards/isa-iec-62443-series-of-standards>, 2026.
- [4] National Institute of Standards and Technology, “The NIST Cybersecurity Framework (CSF) 2.0,” <https://www.nist.gov/publications/nist-cybersecurity-framework-csf-20>, 2024, accessed 2026.
- [5] European Commission, “NIS2 Directive: Securing Network and Information Systems,” <https://digital-strategy.ec.europa.eu/en/policies/nis2-directive>, 2026.